

1から学ぶ JavaScript !

クイズアプリを作ってみよう

今日のゴール： 自分だけの3択クイズアプリ を作る

- 問題と3つの選択肢が表示される
- 選択肢をクリック → 正解 / 不正解がわかる
- 「次の問題へ」で進む
- 全問終了 → スコア表示！

第1問 / 全3問

日本で一番高い山は？

富士山

北岳

奥穂高岳

Webページは 3つの技術 でできている

技術	役割
HTML	構造
CSS	見た目
JavaScript	動き

今日さわるのは **JavaScript**。ページに「動き」をつけていきます！

準備

1. ブラウザで StackBlitz のテンプレートを開く
<https://stackblitz.com/edit/web-quiz-2026?file=index.html>
2. 左に HTML / CSS / JS のファイル、右にプレビュー
3. さわるのは `script.js` が中心。 `index.html` も3回だけ書き足します
4. `debug.js` `styles.css` は用意ずみ。開かなくて大丈夫です

テンプレートは「1問目が表示された状態」から始まります。

今日の進め方

1. JavaScript入門
2. 1問だけのクイズを作る
3. 問題の数を増やす
4. スコアと結果を表示する
5. 自分だけのクイズにする

スライドの見かた

右上に「記述」バッジ → 手を動かしてコードを書くスライド

ハイライトあり — その部分だけ書き足す

```
<button id="choice-0" onclick="alert('クリックされた!')">富士山</button>
```

ハイライトなし — コードをそのまま写す

```
function checkAnswer(selected) {  
  alert('あなたは ' + selected + ' 番を選びました!');  
}
```

デバッグ情報パネルの見かた

プレビュー右上の「デバッグ情報」をクリックすると開きます。自力で原因が分からないときの手がかり用です。

- **章ごとのテスト** — 章の項目が✓になっていれば順調
- **診断** — エラーや書きまちがいの心当たりを表示
- **コピー** — 詰まったら押して、チャットに貼って質問
- エラー画面に変わるとパネルは消えます。同じ内容が **Console** にも出ています



JavaScript入門

クイズを作る前に、JSの基本を知ろう

JavaScript は、ページを「見る人とやりとりできる」ものにする

- ボタンを押したら反応する
- 文字や色を書き換える
- 計算する・判定する

JavaScript の基本を6つ、順番に見ていきます。

① 変数 — データに名前をつけて覚えておく

得点を、結果発表までずっと覚えておきたい

変数 = データに名前をつけて覚えておく仕組み

```
let score = 0;           // score という名前で 0 を覚えておく
score = score + 1;       // 中身を書き換える
console.log(score);      // 名前を取り出す → 1
```

① 変数 — `const` と `let`

あとで書き換えるかどうかで使い分ける

- 得点 → 答えるたびに増える → `let`
- 問題数 → ずっと変わらない → `const`

```
let score = 0;      // 得点（あとで増える）
const total = 3;    // 問題数（変わらない）

score = score + 1;  // OK
total = total + 1;  // まちがい！ 得点と書きまちがえて問題数を増やしてしまった
```

- `const` なら → その場でエラーが出て、書きまちがいにすぐ気づける
- もし `let` にしていたら → エラーにならず動き続け、「全4問」と表示されてしまう

① 変数 — データには種類（型）がある

種類	書き方	できること
数値	18	計算できる
文字列	'鯖江'	文字として表示・つながられる

文字列は `'...'` で囲む（`"..."` でも同じ意味）。囲まないと「変数の名前」として扱われる。

型はほかにもある（`true` / `false` の真偽値など）。今日使うのはこの2つだけ。

```
const a = 'score';    // 文字列の 'score'
const b = score;       // 変数 score の中身
```

`console.log()` で中身を見る

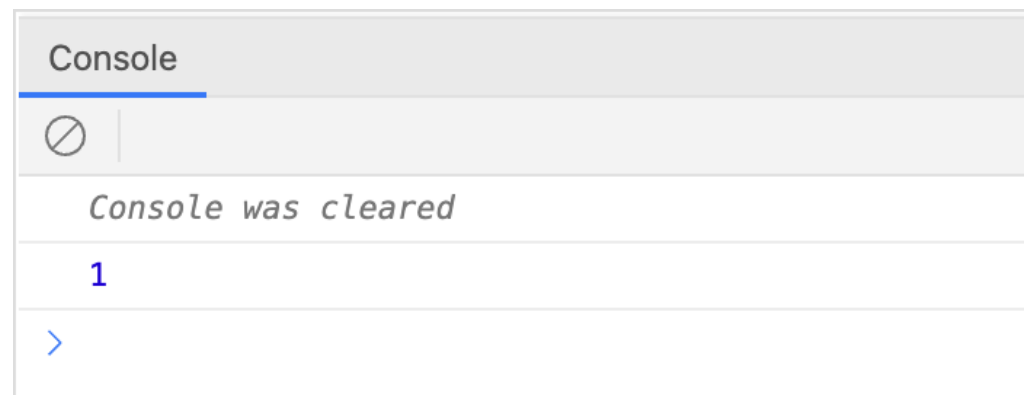
画面に出ない値を確認したい

`console.log(値)` = その値をコンソールに出す命令

見かた — プレビューの下の「Console」（閉じていたらクリックで開く）

```
let score = 0;  
score = score + 1;  
console.log(score); // → Console に 1 と出る
```

分からなくなったら、まず中身を見る。今日も何度か使います。



② 文字列の結合 — `+` でつなげる

「2問正解！」のような表示を作りたい

```
const hello = 'こんにちは' + '！';    // → 'こんにちは！'

const score = 2;
const result = score + '問正解！';    // → '2問正解！'（数値は文字列に変わる）
```

注意 — 数値のつもりが文字列だと、計算されずにつながる

```
console.log('18' + 1);    // → '181'（文字列としてつながる）
console.log(18 + 1);       // → 19（数値として計算）
```

③ 関数 — 処理に名前をつけてまとめる

同じ処理を、何度も使いたい

- 毎回書くと長い。直すときは全部直すことになる
- 処理に名前をつけてまとめ、呼び出すだけにする = 関数

```
function sayHello() {  
  console.log('こんにちは！');  
}
```

```
sayHello();    // → 'こんにちは！' (名前で呼び出すと実行される)
```

③ 関数 — 引数

一部だけ変えて、同じ処理を使いたい

呼び出すときに () で値を渡す = 引数。渡した値で結果が変わる。

```
function sayHello(name) {  
  console.log('こんにちは、' + name + 'さん!');  
}
```

```
sayHello('太郎');    // → こんにちは、太郎さん!  
sayHello('花子');    // → こんにちは、花子さん!
```

④ 条件分岐 — 「もし～なら」で処理を分ける

正解なら○、まちがいなら× — 状況で動きを変えたい

```
if (条件) {  
    // 条件が成り立つときの処理  
} else {  
    // 成り立たないときの処理  
}
```

④ 条件分岐 — 条件は「比較」で書く

比較演算子 = 2つの値を比べる記号。比べた結果が `if` の条件になる。

記号	意味
<code>===</code>	等しい
<code>!==</code>	等しくない
<code>></code> <code><</code> <code>>=</code> <code><=</code>	大小の比較

```
if (selected === 0) { // selected は 0 と等しい？
  console.log('0番が選ばれた！');
}
```

`=` (代入) と `===` (比較) は別物！

⑤ 繰り返し — 同じ処理を何回も

回数分コピーして書きたくない

for = 「〇回くり返して」と1回で書く

```
for (let i = 0; i < 回数; i++) {  
    // くり返す処理 (i は 0, 1, 2, ... と増える)  
}
```

⑤ 繰り返し — 例

「第1問」～「第3問」を順番に表示したい

```
for (let i = 0; i < 3; i++) {  
  const number = i + 1;  
  console.log('第' + number + '問'); // → 第1問, 第2問, 第3問  
}
```

書き方	意味
<code>let i = 0</code>	カウンターを 0 から始める
<code>i < 3</code>	成り立つ間くり返す
<code>i++</code>	1回ごとに 1 増やす

⑥ 配列 — データを順番に並べたリスト

選択肢3つを、変数3個で持ちたくない

番号	0	1	2
中身	'富士山'	'北岳'	'奥穂高岳'

```
const choices = ['富士山', '北岳', '奥穂高岳'];  
  
console.log(choices[0]); // → '富士山'  
console.log(choices[2]); // → '奥穂高岳'
```

[番号] で取り出す。番号は 0 から数える。

⑥ オブジェクト — 名前付きのデータのまとめ

1問分 = 「問題文・答え」のセットで持ちたい

名前（プロパティ）	question	answer
中身（値）	'日本で一番高い山は？'	0

```
const quiz = { question: '日本で一番高い山は？', answer: 0 };
```

```
console.log(quiz.question); // → '日本で一番高い山は？'  
console.log(quiz.answer);  // → 0
```

.名前 で取り出す。

Chapter 1

1問だけのクイズを作ろう

まずはいちばん小さいクイズから。

選択肢を押したら、正解かどうかわかる ようにします。

1-1. テンプレートを見てみよう

いまは1問目が表示されているだけ。ボタンを押しても何も起きません。

この章では、**ボタンを押したら正解 / 不正解が表示される** ようにします。

第1問 / 全3問

日本で一番高い山は？

富士山

北岳

奥穂高岳

1-1. 画面は5つの部品でできている

第1問 / 全3問

① 問題番号

日本で一番高い山は？

② 問題文

富士山

北岳

奥穂高岳

③ 選択肢ボタン

(最初は空)

④ 結果の表示欄

次の問題へ

⑤ 次へボタン

第1問 / 全3問

日本で一番高い山は？

富士山

北岳

奥穂高岳

1-1. HTML (index.html) との対応

`id` は、HTML の要素につける名前です。あとで JS から「どの要素か」を指定するときに、この名前を使います。

```
<!-- ① 問題番号 -->
<p id="question-number">第1問 / 全3問</p>

<!-- ② 問題文 -->
<h1 id="question">日本で一番高い山は？</h1>

<!-- ③ 選択肢ボタン × 3 -->
<div id="choices">
  <button id="choice-0">富士山</button>
  <button id="choice-1">北岳</button>
  <button id="choice-2">奥穂高岳</button>
</div>

<!-- ④ 結果の表示欄（最初は空） -->
<p id="result"></p>

<!-- ⑤ 次へボタン -->
<button id="next-btn">次の問題へ</button>
```

```
<button class="timer-btn" data-  
delta="-60">-</button>
```

```
<button class="timer-btn" data-  
delta="60">+</button>
```

1-2. ボタンを押したら、メッセージを出そう

onclick — クリックされたら、書いたJS を実行する

書く場所 — `index.html` の1つ目のボタン

```
<button id="choice-0" onclick="alert('クリックされた!')">富士山</button>
```

alert = メッセージをポップアップで表示する命令

`'...'` なのは外側の `"` とぶつからないため (JS では `"` も `'` も使える)

成功 — 「富士山」を押すとポップアップが出る。この仕組み = **イベント処理**

```
<button class="timer-btn" data-  
delta="-60"></button>
```

```
<button class="timer-btn" data-  
delta="60"></button>
```

1-3. どのボタンが押されたか、分かるようにしよう

3つのボタンから同じ関数を呼び、押されたボタンの番号を引数で渡す

index.html — 3つのボタンに `onclick="checkAnswer(番号)"` を書き足す

```
<button id="choice-0" onclick="checkAnswer(0)">富士山</button>  
<button id="choice-1" onclick="checkAnswer(1)">北岳</button>  
<button id="choice-2" onclick="checkAnswer(2)">奥穂高岳</button>
```

script.js — 受け取った番号を、まず `alert` で確認する

```
function checkAnswer(selected) {  
  alert('あなたは ' + selected + ' 番を選びました！');  
}
```

1-3. 動作チェック

成功 — 押すボタンで、出てくる番号（0・1・2）が変わる

正解 / 不正解の表示はまだ。ここまでに「押されたボタンの番号を JS が受け取れる」形ができました。表示は次の 1-4 で作ります。

<button class="timer-btn" data-
delta="1-60"></button>

<button class="timer-btn" data-
delta="60"></button>

1-4. 正解 / 不正解を、ページの上に表示しよう

`document.getElementById()` — id を指定して、HTML の要素を取得する

書く場所 — `script.js` の `checkAnswer` を丸ごと書き換え（alert をやめて、④ 結果の表示欄へ）

```
function checkAnswer(selected) {  
  const resultEl = document.getElementById('result');  
  
  if (selected === 0) {  
    resultEl.textContent = '正解！すごい！';  
  } else {  
    resultEl.textContent = '不正解... 正解は「富士山」でした';  
  }  
}
```

1-4. 動作チェック

成功 — ボタンを押すと、ページに正解 / 不正解が表示される

1. `getElementById('result')` で ④ 結果の表示欄を取得
2. `.textContent` で文字を書き換え
3. `if` と `===` で正解を判定

id で要素を取得して書き換える = **DOM操作**

いまは問題が1問だけなので「0番が正解」と決めうちしています。Chapter 2 でデータを見て判定するように直します。

第1問 / 全3問

日本で一番高い山は？

富士山

北岳

奥穂高岳

正解！

次の問題へ

Chapter 2

問題の数を増やそう

いまは1問目が HTML に直接書いてあるだけ。

問題を **データ** にして、何問でも出せるようにします。

<button class="timer-btn" data-delta="60"></button> <button class="timer-btn" data-delta="60">+</button>

2-4. タイズデータを用意しよう

1問分をオブジェクトに、問題ぜんぶを配列にまとめる

書く場所 — `script.js` の先頭

```
const quizData = [
  { question: '日本で一番高い山は？',
    choices: ['富士山', '北岳', '奥穂高岳'], answer: 0 },
  { question: 'jig.jp の本社がある福井県の市は？',
    choices: ['鯖江市', '福井市', '敦賀市'], answer: 0 },
  { question: 'Webページの「動き」を担当する言語は？',
    choices: ['HTML', 'CSS', 'JavaScript'], answer: 2 }
];

let currentQuestion = 0;
let score = 0;
```

2-1. データの形

書き方	意味
<code>quizData</code>	問題をまとめた 配列
<code>{ question, choices, answer }</code>	1問ぶんの オブジェクト
<code>choices</code>	選択肢の配列（今回は3つに統一）
<code>answer</code>	正解の番号（0から数える）

- `quizData` → データを入れ替えない → `const`
- `currentQuestion`（今何問目か）・ `score`（得点） → 進むたびに変わる → `let`

2-1. 動作チェック

画面は変わりません。 `console.log()` で中身を見ます

書く場所 — `script.js` のいちばん下（確認できたら消してOK）

```
console.log(quizData.length);           // → 3  
console.log(quizData[0].question);      // → 日本で一番高い山は？
```

成功 — プレビュー下の **Console** に、上のコメント通りの2行が出る

- 何も出ない → 保存し忘れ、または `script.js` 以外に書いている
- 赤いエラーが出る → カンマや `{ }` `[ ]` の閉じ忘れ（エラーの行番号を見る）

2-2. 問題を表示する関数をつくろう

`showQuestion()` — 「今の問題」を画面に反映する

やることは3つ。ぜんぶ 1-4 で使った DOM操作です。

1. `quizData[currentQuestion]` で今の問題を取り出す
2. 問題番号・問題文・選択肢の文字を書き換える（`choice-0` の id と `choices[0]` の番号をそろえる）
3. 前の問題の結果表示を消し、次へボタンを隠す

`.style.display = 'none'` = その要素を画面から消す（`'inline-block'` で戻る）

関数は呼ばれてはじめて動くので、最後に `showQuestion();` と書いて呼び出すところまでやります。

<button class="timer-btn" data-delta="60">showQuestion()</button> <button class="timer-btn" data-delta="60">+</button>

書く場所 — script.js に追加。最後の1行が呼び出し

```
function showQuestion() {
  const quiz = quizData[currentQuestion];    // 今の問題
  const number = currentQuestion + 1;        // 問題番号

  document.getElementById('question-number').textContent =
    '第' + number + '問 / 全' + quizData.length + '問';
  document.getElementById('question').textContent = quiz.question;

  document.getElementById('choice-0').textContent = quiz.choices[0];
  document.getElementById('choice-1').textContent = quiz.choices[1];
  document.getElementById('choice-2').textContent = quiz.choices[2];

  document.getElementById('result').textContent = '';          // 前の結果を消す
  document.getElementById('next-btn').style.display = 'none';  // 次へを隠す
}

showQuestion();    // ページを開いたら最初の問題を表示
```

2-2. 動作チェック

画面が `quizData` から作られるようになりました

成功 — 次の2つがそろっている

1. 1問目の問題文・選択肢が表示される（見た目は同じでも、出どころが HTML から `quizData` に変わった）
2. 「次の問題へ」ボタンが消えている（`showQuestion` が隠したため）

自分で確かめる — `quizData` の1問目の `question` を好きな文に書き換えて保存。画面の文字も変われば、データから作られている証拠

変わらないとき — プレビュー下の **Console** の赤いエラーを見る。 `Unexpected token` は `( )` `{ }` `'` の閉じ忘れ

<button class="timer-btn" data-delta="60">-</button> 2-3. 正解判定をアップデートしよう <button class="timer-btn" data-delta="60">+</button>

「0番が正解」の決めうちを、`quiz.answer` との比較に変える

書く場所 — `script.js` の `checkAnswer` を丸ごと置き換え（前のものは消す）

```
function checkAnswer(selected) {
  const quiz = quizData[currentQuestion];
  const resultEl = document.getElementById('result');

  if (selected === quiz.answer) {
    resultEl.textContent = '正解!';
    score = score + 1;
  } else {
    const correctText = quiz.choices[quiz.answer];
    resultEl.textContent = '不正解... 正解は「' + correctText + '」';
  }

  document.getElementById('next-btn').style.display = 'inline-block';
}
```

2-3. 動作チェック

成功 — 次の2つがそろっている

1. 正解の選択肢を押すと「正解!」、ほかを押すと正解の名前入りで「不正解...」が出る
2. 答えると「次の問題へ」ボタンが現れる（押してもまだ動きません。次の章で作ります）

自分で確かめる — `score` は画面に出ないので `console.log()` で見る

```
document.getElementById('next-btn').style.display = 'inline-block';  
console.log(score);    // ← checkAnswer の最後に一時的に追加
```

正解を押したときだけ増えれば OK。連打すると増え続けますが、いまは正常です（応用課題で止められます）。

Chapter 3

スコアと結果を表示しよう

解き終わったら「何問正解だったか」を知りたいですね。

「次へ」で進んで、最後にスコアが出る ようにします。

<button class="timer-btn" data-delta="60">3-1. 「次の問題へ」を動かそう</button> <button class="timer-btn" data-delta="60">+</button>

script.js — `nextQuestion` を追加（押すたびに1問進め、最後まで行ったら結果へ）

```
function nextQuestion() {  
  currentQuestion = currentQuestion + 1;  
  
  if (currentQuestion < quizData.length) {  
    showQuestion();    // まだ問題がある → 次を表示  
  } else {  
    showResult();      // 最後まで終わった → 結果へ  
  }  
}
```

index.html — 次へボタンに `onclick` を書き足す

```
<button id="next-btn" onclick="nextQuestion()">次の問題へ</button>
```

成功 — 2問目・3問目に進める（最後の「次へ」で `showResult is not defined` のエラーが出ますが正常。次で作ります）

<button class="timer-btn" data-

3-2. 結果画面をつくろう

<button class="timer-btn" data-

delta="60">+</button>

全問終わったら「○問中○問正解！」を出す

書く場所 — script.js に showResult を追加

```
function showResult() {  
  document.getElementById('question-number').textContent = '結果発表！';  
  
  document.getElementById('question').textContent =  
    quizData.length + '問中 ' + score + '問正解！';  
  
  document.getElementById('choices').style.display = 'none';  
  document.getElementById('next-btn').style.display = 'none';  
  document.getElementById('result').textContent = 'おつかれさま！';  
}
```

成功 — 全問正解で「3問中 3問正解！」、全問まちがえて「3問中 0問正解！」（2回目はプレビューを再読み込み）

完成！

3択クイズアプリが動きました！

おめでとうございます 🎉

結果発表！

3問中 2問正解！

おつかれさま！

Chapter 4

自分だけのクイズにしよう！

仕組みはもう全部できています。
あとは **問題データ**を入れ替えるだけ。

4. オリジナルクイズの作り方

quizData を自分の問題に書き換えるだけ

1問の形 — この3点セットを `,` で区切って並べる

```
{  
  question: 'ここに問題文を書く',  
  choices: ['選択肢1', '選択肢2', '選択肢3'],  
  answer: 0      // 正解の番号 (0から数える)  
}
```

問題はいくつ増やしてもOK。

成功 — 書き換えた問題が1問目から表示され、最後まで解けて「○問中○問正解！」が出る

アイデア — 推しクイズ / 地元クイズ / 学校クイズ / IT雑学 / グルメクイズ

応用課題

余裕がある人はチャレンジ！

応用①：選択肢の数を自由にする（N択）

選択肢のボタンを `for` で自動生成 → 2択でも4択でも出せる

置き換え場所 — `showQuestion` の選択肢3行（`choice-0` ～ `choice-2`）

```
let buttonsHTML = '';
for (let i = 0; i < quiz.choices.length; i++) {
  buttonsHTML +=
    '<button id="choice-' + i + '" onclick="checkAnswer(' + i + ')">'
    + quiz.choices[i] + '</button>';
}
document.getElementById('choices').innerHTML = buttonsHTML;
```

`innerHTML` = 要素の中身を HTML ごと書き換える（`textContent` は文字だけ）

成功 — これまで通り3択で遊べる。`choices` を4つにすると、ボタンも4つに増える

応用②：結果メッセージを変える

得点に応じて、結果のメッセージを変える

書く場所 — `showResult` の最後に追加

```
const resultEl = document.getElementById('result');
const half = quizData.length / 2;

if (score === quizData.length) {
  resultEl.textContent = 'パーフェクト！天才！';
} else if (score >= half) {
  resultEl.textContent = 'なかなかやるね！';
} else {
  resultEl.textContent = '次はもっといけるはず！';
}
```

`else if` で「3つ以上の場合分け」ができます。

成功 — 全問正解と全問不正解でわざと解いて、結果のメッセージが変わる

応用③：もっとチャレンジ

難度	課題	ヒント
★	もう一度チャレンジボタン	<code>currentQuestion</code> と <code>score</code> を 0 に戻す
★	正解した選択肢を緑にする	<code>.style.background = '#4caf50'</code>
★★	一度答えたら押せなくする（いまは連打で得点が増える）	答えたかを覚える変数 + <code>checkAnswer</code> の最初で <code>return</code>
★★	選択肢シャッフル	<code>answer</code> の番号も一緒に変える
★★	タイマー機能	<code>setInterval</code> + <code>clearInterval</code>
★★★★	画像つきクイズ	<code></code> タグを動的に生成

困ったときは

エラーが出たら、プレビュー下の **Console** で赤いメッセージを確認しましょう。

よくあるミス	正しい書き方
<code>=</code> と <code>===</code> の混同	比較は <code>===</code> 、代入は <code>=</code>
カッコの閉じ忘れ	<code>()</code> <code>{}</code> <code>[]</code> は必ずペア
クォーテーション閉じ忘れ	<code>"..."</code> <code>'...'</code> はペア
スペルミス	<code>getElementById</code> の大文字小文字
カンマ忘れ	配列・オブジェクトの要素間

今日学んだこと

概念	使った場面
変数 (<code>const</code> / <code>let</code>)	クイズデータ、スコア管理
文字列の結合 (<code>+</code>)	問題番号・結果の表示
関数と引数 (<code>function</code>)	<code>showQuestion</code> , <code>checkAnswer</code> など
条件分岐 (<code>if</code> / <code>else</code>)	正解判定
繰り返し (<code>for</code>)	応用課題の選択肢生成
配列・オブジェクト	クイズデータの管理
DOM操作 (<code>getElementById</code>)	id で要素を取得して書き換え
イベント処理 (<code>onclick</code>)	ボタンクリック時の処理

```
<script> document.querySelectorAll('.timer-box').forEach(box => { const el =
```